DS homework programs exercise 7 (2025115100)

1. Write a C program to construct an expression tree for a given postfix expression.

```
 #include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* create(char ch) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = ch;
    n->left = n->right = NULL;
    return n;
}

int main() {
    char exp[] = "ab-cd*+";
    struct Node *stack[100], *n;
    int top = -1;

    for (int i = 0; exp[i] != '\0'; i++) {
        n = create(exp[i]);

        if (exp[i] == '+' || exp[i] == '-' ||
            exp[i] == '*' || exp[i] == '/') {
            n->right = stack[top--];
            n->left = stack[top--];
        }
```

```
        stack[++top] = n;
    }

    printf("Expression tree constructed successfully");
    return 0;
}
```

Output

Expression tree constructed successfully

2. Check Whether a Binary Tree Is a BST

```
#include <stdio.h>
#include <limits.h>

struct Node {
    int data;
    struct Node *left, *right;
};

int isBST(struct Node *root, int min, int max) {
    if (root == NULL)
        return 1;

    if (root->data <= min || root->data >= max)
        return 0;

    return isBST(root->left, min, root->data) &&
           isBST(root->right, root->data, max);
}

int main() {
    printf("Use isBST(root, INT_MIN, INT_MAX) to check BST");
```

```
    return 0;
}
```

Output

Use isBST(root, INT_MIN, INT_MAX) to check BST

3. Write a C Program to Implement a Max Heap

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 1; i < n; i++) {
        j = i;

        while (j > 0 && a[(j - 1) / 2] < a[j]) {
            temp = a[j];
            a[j] = a[(j - 1) / 2];
            a[(j - 1) / 2] = temp;
            j = (j - 1) / 2;
        }
    }

    printf("Max Heap: ");
    for (i = 0; i < n; i++)
```

```
        printf("%d ", a[i]);

    return 0;
}
```

Output

Enter number of elements: 5

Enter elements: 10 20 15 30 40

Max Heap: 40 30 15 10 20

4. Write a C program to check whether a cycle exists in an undirected graph.

```
#include <stdio.h>

int graph[10][10], visited[10], n;

int dfs(int v, int parent) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i]) {
            if (!visited[i]) {
                if (dfs(i, v))
                    return 1;
            }
            else if (i != parent)
                return 1;
        }
    }

    return 0;
}
```

```
int main() {
    int e, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &e);

    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = graph[v][u] = 1;
    }

    if (dfs(0, -1))
        printf("Cycle exists");
    else
        printf("Cycle does not exist");

    return 0;
}
```

Output 1

```
Enter vertices and edges: 4 4
0 1
1 2
2 3
3 0
Cycle exists
```

Output 2

```
Enter vertices and edges: 4 3
0 1
1 2
2 3
Cycle does not exist
```

5. Write a C program to count the number of islands in a grid.

```
#include <stdio.h>

char a[10][10];
int m, n;

void dfs(int i, int j) {
    if (i < 0 || j < 0 || i >= m || j >= n ||
        a[i][j] == '0')
        return;

    a[i][j] = '0';

    dfs(i + 1, j);
    dfs(i - 1, j);
    dfs(i, j + 1);
    dfs(i, j - 1);
}

int main() {
    int count = 0;

    printf("Enter rows and columns: ");
    scanf("%d%d", &m, &n);

    printf("Enter grid: ");
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            scanf(" %c", &a[i][j]);

    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
            if (a[i][j] == '1') {
                count++;
                dfs(i, j);
            }
        }
    }

    printf("Number of islands: %d", count);
    return 0;
}
```

Output

```
Enter rows and columns: 4 5
Enter grid: 1 1 0 0 1
1 0 0 1 1
0 0 1 0 0
0 0 0 1 1
Number of islands: 4
```

6. Write a C program to find the Minimum Spanning Tree using Kruskal's Algorithm.

```
#include <stdio.h>

struct Edge {
    int u, v, w;
};

int parent[20];

int find(int x) {
    while (parent[x] != x)
        x = parent[x];
```

```
    return x;
}

int main() {
    struct Edge e[100], temp;
    int n, m, cost = 0, count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++)
        scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);

    for (int i = 0; i < n; i++)
        parent[i] = i;

    for (int i = 0; i < m - 1; i++)
        for (int j = i + 1; j < m; j++)
            if (e[i].w > e[j].w) {
                temp = e[i];
                e[i] = e[j];
                e[j] = temp;
            }

    for (int i = 0; i < m && count < n - 1; i++) {
        int a = find(e[i].u);
        int b = find(e[i].v);

        if (a != b) {
            parent[a] = b;
            cost += e[i].w;
```

```
        count++;

        printf("%d - %d : %d\n",
            e[i].u, e[i].v, e[i].w);
      }
    }

    if (count == n - 1)
      printf("Minimum cost = %d", cost);
    else
      printf("Spanning tree does not exist");

    return 0;
}
```

Output

```
Enter vertices and edges: 4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost = 19
```

7. Write a C program to find the shortest paths from a source vertex to all other vertices using Dijkstra's Algorithm.

```
#include <stdio.h>

#define INF 9999
```

```
int main() {
    int a[10][10], d[10], visited[10] = {0};
    int n, src, u, min;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 for no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (i != j && a[i][j] == 0)
                a[i][j] = INF;
        }

    printf("Enter source: ");
    scanf("%d", &src);

    for (int i = 0; i < n; i++)
        d[i] = INF;

    d[src] = 0;

    for (int k = 0; k < n; k++) {
        min = INF;
        u = -1;

        for (int i = 0; i < n; i++)
            if (!visited[i] && d[i] < min) {
                min = d[i];
```

```
            u = i;
        }

    if (u == -1)
        break;

    visited[u] = 1;

    for (int v = 0; v < n; v++)
        if (!visited[v] && a[u][v] != INF &&
            d[u] + a[u][v] < d[v])
            d[v] = d[u] + a[u][v];
  }

  printf("Shortest distances:\n");
  for (int i = 0; i < n; i++)
    printf("%d -> %d = %d\n", src, i, d[i]);

  return 0;
}
```

Output

```
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 4 1 0
4 0 2 5
1 2 0 8
0 5 8 0
Enter source: 0
Shortest distances:
0 -> 0 = 0
0 -> 1 = 3
```

0 -> 2 = 1

0 -> 3 = 8

8. Write a C program to find a mother vertex in a directed graph.

```
#include <stdio.h>

int a[10][10], visited[10], n;

void dfs(int v) {
   visited[v] = 1;

   for (int i = 0; i < n; i++)
      if (a[v][i] && !visited[i])
         dfs(i);
}

int main() {
   int e, u, v, candidate = 0;

   printf("Enter vertices and edges: ");
   scanf("%d%d", &n, &e);

   for (int i = 0; i < e; i++) {
      scanf("%d%d", &u, &v);
      a[u][v] = 1;
   }

   for (int i = 0; i < n; i++) {
      if (!visited[i]) {
         dfs(i);
         candidate = i;
```

```
        }
    }

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    dfs(candidate);

    for (int i = 0; i < n; i++)
        if (!visited[i]) {
            printf("No mother vertex");
            return 0;
        }

    printf("Mother vertex: %d", candidate);
    return 0;
}
```

Output

```
Enter vertices and edges: 5 6
0 1
0 2
1 3
2 3
3 4
4 0
Mother vertex: 0
```

9. Write a C program to find Strongly Connected Components using Kosaraju's Algorithm.

```
#include <stdio.h>

int a[10][10], t[10][10], visited[10];
int stack[10], top = -1, n;
```

```
void dfs1(int v) {
   visited[v] = 1;

   for (int i = 0; i < n; i++)
      if (a[v][i] && !visited[i])
         dfs1(i);

   stack[++top] = v;
}

void dfs2(int v) {
   visited[v] = 1;
   printf("%d ", v);

   for (int i = 0; i < n; i++)
      if (t[v][i] && !visited[i])
         dfs2(i);
}

int main() {
   int e, u, v;

   printf("Enter vertices and edges: ");
   scanf("%d%d", &n, &e);

   for (int i = 0; i < e; i++) {
      scanf("%d%d", &u, &v);
      a[u][v] = 1;
      t[v][u] = 1;
   }
```

```
    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs1(i);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("Strongly Connected Components:\n");

    while (top >= 0) {
        v = stack[top--];

        if (!visited[v]) {
            dfs2(v);
            printf("\n");
        }
    }

    return 0;
}
```

Output

```
Enter vertices and edges: 5 5
0 1
1 2
2 0
1 3
3 4
Strongly Connected Components:
0 2 1
3
```

4

10. Write a C program to determine whether a given graph is bipartite.

```
#include <stdio.h>

int a[10][10], color[10], n;

int bfs(int s) {
   int q[100], front = 0, rear = 0;

   q[rear++] = s;
   color[s] = 0;

   while (front < rear) {
      int u = q[front++];

      for (int v = 0; v < n; v++) {
         if (a[u][v]) {
            if (color[v] == -1) {
               color[v] = 1 - color[u];
               q[rear++] = v;
            }
            else if (color[v] == color[u])
               return 0;
         }
      }
   }

   return 1;
}
```

```
int main() {
   int e, u, v;

   printf("Enter vertices and edges: ");
   scanf("%d%d", &n, &e);

   for (int i = 0; i < e; i++) {
      scanf("%d%d", &u, &v);
      a[u][v] = a[v][u] = 1;
   }

   for (int i = 0; i < n; i++)
      color[i] = -1;

   for (int i = 0; i < n; i++) {
      if (color[i] == -1 && !bfs(i)) {
         printf("Not Bipartite");
         return 0;
      }
   }

   printf("Bipartite Graph");
   return 0;
}
```

Output 1

```
Enter vertices and edges: 4 3
0 1
1 2
2 3
Bipartite Graph
```

Output 2

Enter vertices and edges: 3 3

0 1

1 2

2 0

Not Bipartite